

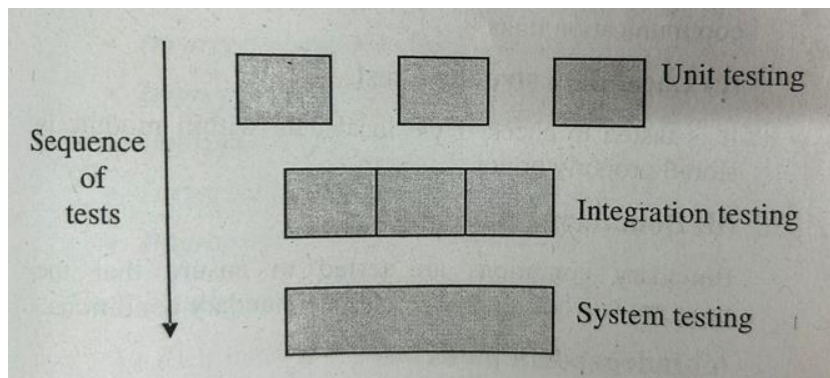
Unit 4

Software Testing:

- Software testing is the process of executing a program with the intention to detect errors in the code.
- It is essentially required before launching of software project.
- Testing is the process of evaluating a system by manual or automated means to verify that it satisfies specified requirements and identify differences between expected and actual results

In order to conduct a proper testing, following order should be performed:

Levels of testing:

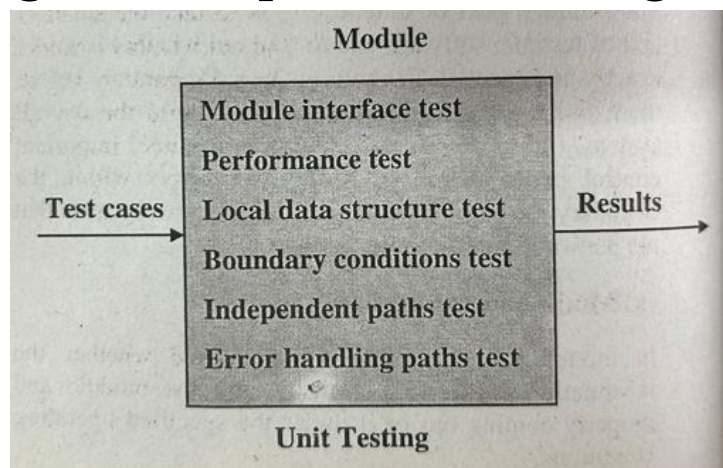


a. Unit Testing:

- It is the process of checking functionality of module.
- A Module is the smallest unit of software.

- It begins after the source code is developed and verified for the correct syntax.
- The primary goal of unit testing is to take the smallest unit of testable software and to find out whether it works exactly as expected.
- Each unit is tested separately before these units are integrated together to build the overall system.

Following tests are performed during Unit Testing:



- **Module Interface Test:** In module interface test it is checked whether information is properly flowing into the module and properly coming out of it under the specified operating conditions.
- **Performance Test:** It is designed to verify response time, execution time, throughput and traffic rates on data channels.
- **Local data structure test:** It is tested to check if local data within module is stored properly or not.

- **Boundary conditions test:** Boundary conditions are tested to ensure that the program is working properly at the boundary conditions.
- **Independent paths test:** Independent paths are tested to check that they are executing their task properly or not.
- **Error handling paths test:** These tested to check whether errors handling is working properly or not.

Unit Testing Procedure:

- ☐ As each module is tested individually in unit testing so there is need to obtain data from other module or pass data to other module.
- ☐ This is achieved by using stubs and drivers.
- ☐ The driver simulates a calling unit and the stub simulates a called unit.
- ☐ A driver is a program that takes test case data and passes it to the module being tested.
- ☐ Stubs are also programs used to replace modules that are subordinate to the module being tested.
- ☐ Unit testing requires drivers and stubs to be written.
- ☐ This overhead can be reduced if drivers and stubs are kept simple.

b. Integration Testing:

- The next step of testing is integration testing.

- In this, many units tested modules are combined into subsystems which are then tested.
- The main purpose of unit testing is to find out that each independent module is correctly implemented.
- But unit test does not guarantee that these modules will work fine when integrated together as system.
- Therefore, Integration Testing must be performed.
- Integration Testing is a systematic technique that integrate tested modules, find errors, remove them and check program structure as specified by design.

Following Types of error arise during integration of testing modules:

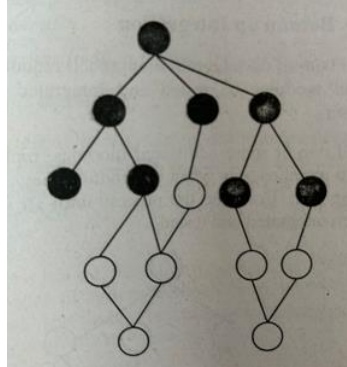
- Global data can create problems.
- Data can be lost across an interface i.e. data coming out of a module is not going to the desired module.
- Modules when combined do not produce the required major module.
- One module can have an adverse effect on another.

Three Basic Types of Integration are:

i) Top-down Integration:

- Top-down Integration is an incremental approach that requires highest-level modules tested and integrated first.

- Add one or more other modules, continue process until all modules have been integrated and tested.
- High level logic and data flow is tested early in the process and thus minimizes the need for drivers.



Advantages of Top-down Integration:

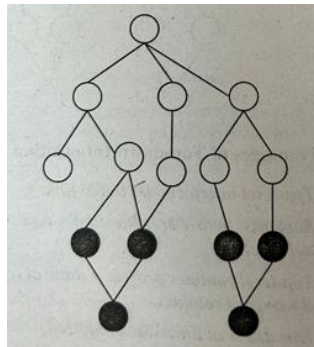
- Top-level interfaces are tested first.
- Supports both breadth first and Depth first approaches.
- Top-level routines provide a natural test for lower-level routines.
- One driver at the most is required.

Disadvantages of Top-down Integration:

- Delays verification of low-level modules.
- Not enough data in stubs to feedback to calling module.
- Provides poor support for early release of limited functionality.
- Need for stubs complicates test management.

ii). Bottom up Integration:

- Bottom up integration requires lowest level modules tested and integrated first using driver.
- Add more additional modules, replacing drivers with modules only when all modules they call have been integrated.
- Continue until all modules integrated and tested.



Advantages of Bottom up Integration:

- Stubs are not required as developing and testing starts with the actual modules.
- Allows early verification of low-level modules.

Disadvantages of Bottom up Integration:

- Need for drivers complicates test management.
- Provides poor support for early release of limited functionality.
- Delays verification of high level behaviour.

iii) Mixed Integration Testing

- A mixed integration testing is also called sandwiched integration testing.

- A mixed integration testing follows a combination of top down and bottom-up testing approaches.
- In top-down approach, testing can start only after the top-level modules have been coded and unit tested. In bottom-up approach, testing can start only after the bottom level modules are ready.
- This mixed approach overcomes this shortcoming of the top-down and bottom-up approaches.
- It is also called the hybrid integration testing.
- Also stubs and drivers are used in mixed integration testing.

Advantages of Mixed Integration Testing:

- Mixed approach is useful for very large projects having several sub projects.
- This approach overcomes shortcoming of the top-down and bottom-up approaches.
- Parallel test can be performed in top and bottom layer tests.

Disadvantages of Mixed Integration Testing:

- For mixed integration testing, it requires very high cost because one part has a Top-down approach while another part has a bottom-up approach.
- This integration testing cannot be used for smaller systems with huge interdependence between different modules

c. System Testing:

- System Testing implies testing of the entire software.
- The goal of system testing is to see if the software meets its requirements.
- System testing includes a series of different tests to fully exercise the computer-based system.
- Each test has a different purpose and the series of tests verify that all system elements are properly integrated and are performing the allocated functions.

System Testing involves following types of testing:

i)Recovery Testing:

- Recovery Testing is designed to examine how easily and completely the system can recover a system failure.
- Many computer-based systems recover from faults and resume operation within a pre-specified.
- A system with quick recovering capability and with minimum human intervention is desirable.

ii)Security Testing:

- Protection mechanism built into a system to determine that the system is protected from unauthorized personnel.

- Other systems cannot gain access to the system and information within it.

iii) **Stress Testing:**

- Stress Testing test the system performance in abnormal situations.
- Stress Testing executes a system in a manner that demands resources in abnormal quantities.

Role of Software Testing during SDLC

Software Development Phase	Testing
Requirement Specification	• To identify the test strategy. • To check the sufficiency of requirements. • To create functional test conditions.
Design	• To check the consistency of design with the requirements. • To check the sufficiency of design. • To create structural and functional test conditions.
Coding	• To check the consistency of implementation with the design. • To check the sufficiency of implementation. • To create structural and functional test conditions for programs/units.
Testing	• To check the sufficiency of the test plan. • To test the application programs.
Installation and Maintenance	• To put the tested system under operation. • To make changes in the system and retest the modified system.

Design of Test Cases:

- A Test Case is a set of instructions designed to discover a particular type of error or defect in the software system by inducing a failure.
- A Test Case is a document that has an input, an action and an expected result, to determine if a feature of an application is working correctly.
- The process of developing test cases help to find problems in requirements, design of an application, since it requires completely thinking of application.
- Each test case needs proper documentation, preferably in a fixed format.
- The goal of test case selection is:
 - To select the test cases such that the maximum possible number of faults are detected by the minimum possible number of test cases.
 - To ensure that there is no error in the program and if it is so then it should be immediately depicted.

Designing / Writing good Test Case:

- Well-designed test cases are the most important tools for discovering defects in software.
- Good test cases save time and money and even all the reputation of the organization.

Following points are considered while designing test cases:

1.Simple and Transparent: Create test cases which are as simple as possible. They must be clear and concise as author of test case may not execute them.

2.Create test case with end user in mind: Create test case that meets customer requirements. A tester must create test cases keeping in mind end user perspective.

3.Avoid test case repetition: Do not repeat test cases. If a test case is needed for executing some other test case, call the test case by its test case id.

4.Avoid assuming: While preparing test case, do not assume functionality and features of the software application. Stick to the specification documents.

5.Ensure 100% coverage: Make sure the test cases check all software requirements mentioned in the specification document. Use Traceability Matrix to ensure no function / conditions is left untested.

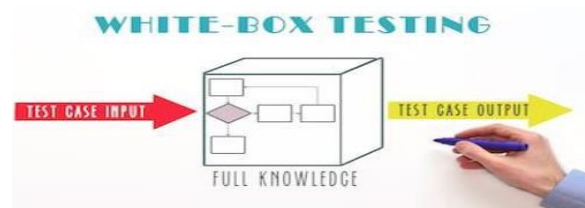
7.Implement Testing Technique: Testing technique must be used for effective testing.

8.Test cases must be identifiable: Name the test case ids such that they are identified easily while tracking defects or identifying a software requirement at a later stage. Use exact and consistent names of forms, fields.

Types of testing

1. Structural Testing / White box testing:

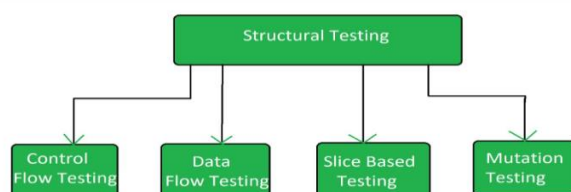
- Structural Testing is also called **white box** or **glass box** testing because in structural testing one requires to know what is happening inside system.
- This testing is the detailed investigation of internal logic and structure of the code.
- Structural testing is performed by the team which knows the development phase of the software.
- This testing is applied to small program units.



Structural Testing tests the following:

- Loops of the procedure - execute all loops at their boundaries and within their operational limits.
- Internal data structures - exercise internal data structures to ensure their validity.
- Decision points - exercise all logical decisions on their true and false sides.

Types of testing methods under Structural Testing:



i)Control Flow Testing:

- This method is used to test the logic of the code so that required result can be obtained.
- The entire code, design and structure of the software have to be known for this type of testing.
- This type of testing is used by the developers to test their own code and implementation.

ii)Data Flow Testing:

- Data flow testing looks at how data moves within a program.
- It focuses on the points at which variable receives values and points at which these values are used.
- It is useful for selecting test paths of a program containing nested if and loop statements.
- This technique is based on observation that values associated with variables effect program execution.

iii)Slice Based Testing:

- It is useful for software debugging, software maintenance, program understanding and quantification of functional cohesion.
- It divides program into different slices and tests that slice which can majorly affect entire software.

iv)Mutation Testing:

- Mutation Testing is performed to design new software tests and also evaluate the quality of already existing software tests.
- Mutation testing is related to modification a program in small ways.
- It focuses to help tester develop effective tests or locate weaknesses in test data used for the program.

Need / Advantage of Structural Testing:

- It helps in optimizing the code.
- It helps in removing extra lines of code, which can bring in hidden defects.
- All the features and functionality within the application can be tested.
- Testing can be started at the very early stage.

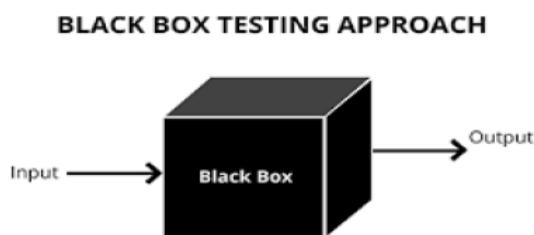
Disadvantage of Structural Testing:

- It requires knowledge of the code to perform test.
- It requires training in the tool used for testing.
- Sometimes it is expensive.

2. Functional Testing / Black box Testing:

- Functional Testing is a way of checking software to ensure that it has all the required functionality that's specified within its functional requirements.
- It is also called black box testing.

- In this software testing the tester does not know the internal working of the product being tested.
- Functional Testing ensures that the requirements are properly satisfied by the application.
- This type of testing is not concerned with how processing occurs, but with results of processing.
- If outputs obtained are same as the expected ones, then the product meets functional requirements. This testing tests overall requirements of product.
- In this approach internal procedures of program are ignored that's why it is called Black-Box Testing.
- Functional Testing is used to verify that software is providing same output as required by end-users.
- Black box testing is conducted at later stages of testing.



Black box testing covers following types of errors:

- **Basic Usability:** To check whether user freely navigate through screens without difficulty.
- **Mainline functions:** This involves testing the main feature and functions of the application.

- **Accessibility:** This involves testing accessibility of the system for the user.
- **Error Conditions:** To check appropriate error messages are displayed in error conditions.

Functional Testing involves the following steps:

- Identify functions that the software is expected to perform.
- Create input data based on the function's specifications.
- Determine the output based on the function's specifications.
- Execute the test case.
- Compare the actual and expected outputs.

Different techniques used in functional Testing:

1. Equivalence Partitioning:

- This method divides input domain of program into classes of data from which test cases can be derived.
- These classes are known as equivalence classes and the process of making equivalence classes is called equivalence partitioning.

2. Boundary Value Analysis:

- In boundary value analysis, rather than selecting any element of an equivalence class, focus is on the selection of test cases at the boundaries values.
- Boundary values include maximum, minimum, just inside/outside boundaries and error values.
- It has been observed that large number of errors tend to occur at boundaries of the input domain.

3. Cause-effect Graphing Techniques:

- Cause-effect graphing is a technique that helps in selecting combinations of input conditions in a systematic manner so that number of test cases do not become large.
- This technique starts with identifying causes and effects of the system under testing.
- A cause is a distinct input condition and an effect is a distinct output condition.
- Each condition is represented by a node in the cause effect graph.

Need / advantages of Functional Testing:

- It is well suited and efficient for large codes.
- Separates user's perspective from the developer's perspective through visibly defined roles.
- Tests are done from a user's point of view.

- Tester can be non-technical.
- The tester need not have detailed functional knowledge of system.

Disadvantages:

- Limited coverage, since only a selected number of test scenarios is actually performed.
- The test cases are difficult to design.

Debugging:

- The detection of errors and removal of those errors is called debugging.
- Debugging means identifying, locating and correcting bugs usually by running the program.
- These bugs are usually logical errors.
- Debugging consist of determining exact nature and location of the suspected error and fixing the error.
- Debugging is that activity which is performed after executing a successful test case.

Process of Debugging

The steps involved in debugging are:

- Problem identification and report preparation.
- Assigning the report to the software engineer defect to verify that it is genuine.

- Defect Analysis using modeling, documentation, finding and testing candidate flaws, etc.
- Defect Resolution by making required changes to the system.
- Validation of corrections.

Debugging Techniques:

1. Brute Force Debugging:

- This debugging is the most common debugging technique through it is least efficient.
- In this debugging technique, the program is always loaded with print statements to always print the common or intermediate values with the hope that a few of the printed values will help to identify the statement with error.

2. Backtracking Debugging:

- Backtracking is common debugging technique.
- This involves backtracking the incorrect results through the logic of the program.
- In Backtracking debugging technique, source code is outlined backwards until error is discovered.
- This debugging approach can be used successfully in small programs.

3. Debugging by induction:

- In this, induction process is used where one moves from the particulars of a situation to the whole.

- This debugging technique is a process that always organizes, locates data, and devises a hypothesis.

4. Debugging by Logging and Tracing:

- Using logging and tracing tools to identify the sequence of events leading up to the error.
- This approach involves collecting and analyzing logs and traces generated by the system during its execution.

5. Debugging by testing:

- This debugging technique involves use of test cases.

Advantages of Debugging

1. **Improved system quality:** By identifying and resolving bugs, software system can be made more reliable and efficient, resulting in improved quality.
2. **Reduced system downtime:** By identifying and resolving bugs, system made more stable and less downtime, result in improved availability for users.
3. **Increased user satisfaction:** By identifying and resolving bugs, software system will better able to meet needs of users, which can result in increased satisfaction.
4. **Reduced development costs:** Identifying and resolving bugs early in development process, can

save time and resources that would be spent on fixing bugs later in the development process.

5. **Increased security:** By identifying and resolving bugs that could be exploited by attackers, software system can be made more secure.
6. **Facilitates change:** With debugging, it becomes easy to make changes to software as it becomes easy to identify & fix bugs caused by the changes.
7. **Better understanding of system:** Debugging help developers to gain better understanding of how a software system works, how different components of the system interact with one another.
8. **Facilitates testing:** By identifying and resolving bugs, it makes it easier to test software and ensure that it meets the requirements and specifications.

Disadvantages of Debugging

1. **Time-consuming:** Debugging can be a time-consuming process, if bug is difficult to find. This can cause delays in the development process.
2. **Requires specialized skills:** Debugging can be a complex task that requires specialized skills and knowledge. This can be a challenge for developers who are not familiar with the tools and techniques.
3. **Can be difficult to diagnose:** Some bugs caused by interactions between different components of a

software system, which can make it challenging to identify root cause of the problem.

4. **Can be difficult to fix:** Some bugs may be caused by architecture issues, which can be difficult to fix without significant changes to the software system.
6. **Limited insight:** Debugging tools only provide limited insight into problem and not provide enough information to identify root cause.
7. **Expensive:** Debugging can be expensive process, it requires additional resources such as specialized debugging tool, additional development time.

Software Maintenance:

- Software Maintenance is the term that is used to refer to the modifications that are made to software system after its release.
- Maintenance is modification of software product after delivery to correct faults, to improve performance or to adapt the product to a modified environment.
- Software Maintenance activity include error detection, correction and optimization.
- Maintenance occurs after:
 - The product is delivered at user's site.
 - Installed
 - It is in operational state.

Reasons / Need of Software Maintenance:

- Changes in user requirement from time to time.
- Advancement / changes in Hardware technologies.
- To improve system efficiency and throughput.
- To maintain quality of software overtime.
- To modify software to correct errors.
- To optimize the code to run faster.
- To make enhancements in existing software to upgrade the performance.
- To eliminate any unwanted side effects resulting from modifications.
- To enhance the understand ability of the code and make it easier to work with.
- To prevent system performance from degrading.

Types of Software Maintenance:

a. Corrective Maintenance:

- Corrective Maintenance refers to a modifications initiated by defects in the software products.
- The main aim of corrective maintenance is to correct any remaining errors in software product.

b. Adaptive Maintenance:

- Adaptive Maintenance means modifying the software product to react to changes in the outside environment in which the system operates.

- Adaptive maintenance is generally not requested by client but it is imposed by outside environment.
- Adaptive Maintenance involves adjusting software to accommodate changes in the environment.

c. Perfective Maintenance:

- Perfective Maintenance means improving:
 - Efficiency
 - Performance
 - Maintainability
 - Effectiveness of the software product.
- Perfective Maintenance involves changing the software to improve its qualities.
- These changes are needed to add new functions, improve performance make it easier to use.

The cost of maintenance is high. The cost of maintenance is as high as 67% of the cost of entire software process cycle because of following reasons-

- Older software work on slow machines with less memory, storage capacity cannot keep themselves against new software on modern hardware.
- As technology advances, it becomes costly to maintain old software.
- Most maintenance engineers are new and use trial and error method to rectify problem.

- Changes made can easily affect original structure of software, making it hard for any other changes.
- Changes are left undocumented which may cause more conflicts in future.
- Structure of Software Program
- Programming Language
- Dependence on external environment
- Staff reliability and availability

Factor techniques to reduce maintenance cost-

1.Application Domain-If application of the program is defined and well understood, system requirements clear so maintenance due to changing needs minimized.

2. Staff Stability-It is simple for the original writer of a program to understand and change an application rather than new person who understand program by study reports and code. So maintenance costs will reduce.

3.Dependence on External Environment-If an application is dependent on its external environment, it must be modified as the climate changes.

For example: Changes in taxation system might need payroll, accounting, stock programs to be modified.

4. Hardware Stability- If an application is designed to operate on a specific hardware configuration and that configuration don't changes during program's lifetime, no maintenance costs due to hardware changes.

Reverse Engineering:

- Reverse Engineering is the process of recreating a design by analyzing a product.
- Reverse Engineering is the process followed in order to find difficult, unknown and hidden information about a software system.
- Reverse Engineering is a process of recovering the design, requirement specifications and functions of a product from an analysis of its code.
- Reverse Engineering is the practice of analyzing a software system, either in whole or in part, to extract design & implementation information.

Objectives of Reverse Engineering:

- Improve documentation shortcomings.
- Handle complexity.
- Generate alternate views.
- Recover lost information.
- Facilitate reuse.
- Software modernization.

Reasons for reverse engineering: -

Reverse engineering is widely for different purposes:

- Identifying components and their interrelationships
- If there is documentation missing for a part

- To inspect equipment and components for quality issues and tolerances
- If there is a dispute over a patent, a company can reverse engineer a competitor's product to prove their design has been stolen
- To make software interoperate more effectively
- When new digital models of components or systems are needed
- To gain greater understanding of what went wrong with an ineffective design
- We can discover how items were originally made – about technology and history that have been lost.
- Analysis of historical or obsolete components allows for improvement, combining older knowledge with new, advanced technology
- Reverse engineering is an important part of obsolescence management. When technology or machinery is still relying on components which are no longer made, reverse engineering can be only way to keep things running without replacing whole system.

Software Re-engineering:

- Re-engineering is the examination and alternation of an existing software system to reconstitute it in a new form.

- Re-engineering means re-implementing system to make them more maintainable.
- It is the investigation and redesign of individual components.
- It is a process which is done to improve the maintainability of a software system.
- It is process of modifying the internal mechanism of a system without changing its functionality.

Objectives of Software Re-engineering are:

- Improve understanding of a software system through analysis.
- Capture and preserve the existing knowledge for a software system through analysis and modelling.
- Preparation for functional enhancement.
- Improve maintainability.
- Increase reusability.
- Improve reliability

Advantages of Software Re-engineering:

- Reduced cost
- Reduced risk
- Better use of existing staff
- Saves time
- Increases reliability

Configuration Management:

- Software configuration management is an activity which is applied throughout software engineering process.
- Configuration management control changes made to system and manages the different versions of the software product.
- Configuration management systematically control the changes that take place during development.
- Thus, the means by which the process of software development & maintenance is controlled is called configuration management.
- It is an art of identifying, organizing and controlling modifications to software.
- The goal is to maximize productivity by minimizing errors.
- Configuration management identifies controls, reports modifications that occur while software is developed and after it has released to customer.

Software Configuration Management activities are:

- Software configuration identification.
- Software configuration control.
- Software configuration status accounting.
- Software configuration auditing.

Importance of Software Configuration Management

- **Effective Bug Tracking:** Linking code modifications to issues that have been reported, makes bug tracking more effective.
- **Continuous Deployment and Integration:** SCM combines with continuous processes to automate deployment and testing, resulting in more dependable and timely software delivery.
- **Risk management:** SCM lowers the chance of introducing critical flaws by assisting in the early detection and correction of problems.
- **Support for Big Projects:** Source Code Control (SCM) offers an orderly method to handle code modifications for big projects.
- **Reproducibility:** By recording precise versions of code, libraries, and dependencies, source code versioning (SCM) makes builds repeatable.
- **Parallel Development:** SCM facilitates parallel development by enabling several developers to collaborate on various branches at once.

Disadvantages of SCM

1. Increased complexity in large software systems.
2. Difficulty in managing dependencies and ensuring that all changes are properly integrated.

Documentation:

- Software documentation is the information that describes the development, operations and maintenance of the software.
- A good documentation is a sign of the professional pride a programmer puts into software.
- Good documentation increases understand ability and maintainability of a software product.
- In large projects, normally the documentation is done outside the source code.

Types of Software Documentation:

Internal Documentation:

- Internal documentation is done by using comments in the program.
- Comments are textual statements that are meant for the program reader and are not executed.
- The comments should explain what the code is doing not how it is doing.

Internal Documentation provides:

- Help to the programmer for further review of the software and existing system.
- Valuable maintenance if it is properly written and kept consistent with the code.

External Documentation:

External documentation covers the supporting documentation in the form of separate documents.

It is provided through various types of documents:

1. User's manual
2. Software requirements specification document
3. Design document
4. Test document etc.

External documentation provided for data entry operators and computer operators should cover:

- The necessary commands to load the software.
- Types of messages which the software may display during its execution.
- General descriptions of system's functions so that the operator can obtain a feeling for whether or not the system is behaving correctly.

External documentation should cover:

- A detailed description of the function performed by the software.
- The ways of supplying data to the program, the format and contents of data together with any restriction on values.
- Details of error messages.

Advantages of software documentation

- The presence of documentation helps in keeping the track of all aspects of an application and also improves the quality of the software product.
- The main focus is on development, maintenance, and knowledge transfer to other developers.
- Helps development teams during development.
- Helps end-users in using the product.
- Improves overall quality of software product
- It cuts down duplicative work.
- Makes easier to understand code.
- Helps in establishing internal coordination in work.

Disadvantages of software documentation

- The documenting code is time-consuming.
- The document should be updated regularly.
- The documentation has no influence on the performance of an application.